

TASK 02 — Timeout, Retry & Failure Handling

Step 1 — Understand timeout

A **timeout** means:

“I will wait for the other service only for a limited amount of time. If it does not respond within that time, I will stop waiting and return an error.”

Example

Suppose:

Order Service → User Service

Order Service needs user information before creating an order.

If User Service normally responds in **200 ms**, everything is fine.

But if User Service becomes slow:

Order Service

| Request

User Service

| takes too long...

X

Without a timeout, Order Service might wait indefinitely.

With a timeout:

Order Service

| Request

User Service

| Wait 2 seconds

TIMEOUT

↓

Controlled Error

Connection Timeout

This is the maximum time allowed to **establish a connection** with another service.

Example:

Order Service → User Service

| Connecting...

| 5 seconds

Connection Timeout

If the User Service cannot be reached within the configured time, the connection attempt fails.

Example:

Connection timeout = 2 seconds

The client will not keep trying to establish the connection forever.

Read Timeout

The connection is established, but the service does not send a response quickly enough.

Order Service

| Connection established

User Service

| Processing...

| Processing...

| Processing...

Read Timeout

Example:

Read timeout = 3 seconds

If User Service does not provide the expected response within 3 seconds, Order Service stops waiting.

Request Timeout

This is the **maximum overall time allowed for the request**.

For example:

Order Service → User Service

Start



Connection



Send request



User Service processing



Receive response



Finish

If the entire operation takes longer than the configured limit:

Request Timeout

The exact meaning depends on the HTTP client/framework being used, because some libraries distinguish connection, read, and overall request timeouts differently.

Why indefinite waiting is dangerous

This is especially important in microservices.

Suppose:

Order Service



User Service



Database

If User Service becomes slow and Order Service waits forever:

100 requests



Order Service



User Service



Waiting...

Waiting...

Waiting...

Eventually Order Service can run out of:

- Threads
- HTTP connections
- Memory
- CPU resources

Then even healthy requests may start failing.

This can create a **cascading failure**:

Slow User Service



Order Service waits



Order Service resources exhausted



Order Service becomes slow



Other services are affected

So **timeouts** prevent one unhealthy service from blocking the entire system indefinitely.

Step 2 — Configure Timeout

We need to configure a reasonable timeout for this communication.

For example:

Order Service

| HTTP request

User Service

Timeout = 2 seconds

The basic flow is:

Request



Wait maximum configured time



Did User Service respond?



NO



Timeout



Return controlled error

Example

Suppose we configure:

Timeout = 2 seconds

User Service responds in:

500 ms

Result:

SUCCESS

But if User Service responds after:

5 seconds

Order Service does not wait the full 5 seconds.

Instead:

0 sec



Request sent



1 sec



2 sec → TIMEOUT



Controlled error

Controlled Error:

Instead of allowing the application to hang or crash, Order Service should return an appropriate response.

For example:

HTTP 503 Service Unavailable

with a response such as:

```
{  
  "status": 503,  
  "message": "User Service is temporarily unavailable"  
}
```

This is much better than:

Request hanging forever

Step 3 — Test Timeout

User Service wait:

5 seconds

while Order Service has:

Timeout = 2 seconds

Then test:

Client



Order Service



User Service

|---- Wait 5 seconds ----|

But Order Service only waits: 2 seconds

Therefore:

2 seconds



Timeout



Order Service handles error



Client receives controlled response

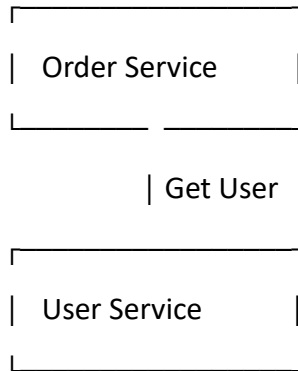
The important thing to verify is:

Order Service must not wait indefinitely for User Service.

Complete Example

Imagine an online shopping application.

Services



Customer requests: POST /orders

Order Service needs user information: GET /users/101

Normal case

Order Service

| GET /users/101

User Service

| 200 OK

Order Service

|

Create Order

Timeout case

Order Service

| GET /users/101

User Service

| Very slow

|

2 seconds reached

|

TIMEOUT

|

Return controlled error

In Spring Boot

Since you're working with **Spring Boot microservices**, a simple example using RestClient could look like this:

```
RestClient restClient = RestClient.builder()
    .baseUrl("http://localhost:8081")
    .requestFactory(new SimpleClientHttpRequestFactory())
    .build();
```

For production applications, you would normally use a client configuration that supports explicit connection/read timeout settings, such as Apache HttpClient or another supported HTTP client.

Example, conceptually:

Connection Timeout = 2 seconds

Read Timeout = 3 seconds

Request Timeout = 5 seconds

The important idea is:

Order Service

| Timeout configured

User Service

Step 4 — Understand Retry

Retry:

A **retry** means sending the same request again after a temporary failure.

Example:

Order Service

| Request

User Service

X Failure

Retry

|

User Service

✓ Success

Instead of immediately giving up after one temporary failure, the application tries again.

Temporary Failure

A temporary failure is a problem that may disappear after a short period.

For example:

Attempt 1 → User Service unavailable

Attempt 2 → User Service unavailable

Attempt 3 → User Service available

The third request succeeds.

Common temporary problems can include:

- Temporary network interruption
- Service temporarily overloaded
- Connection reset
- HTTP 503 Service Unavailable
- Some transient gateway/network errors

The important idea is:

Temporary problem

↓

Wait

↓

Try again

Why is Retry Required?

Without retry:

Order Service



User Service

X Temporary failure



Request Failed

The user may receive an error even though User Service becomes healthy a moment later.

With retry:

Order Service



User Service

X



Retry



User Service

X



Retry



User Service

✓



Success

This can make the application more resilient to **temporary/transient failures**.

Maximum Retry Attempts

You should **never retry forever**.

For example:

Maximum attempts = 3

Then:

Attempt 1 → Failed

Attempt 2 → Failed

Attempt 3 → Failed



Stop retrying



Return error

If the third attempt succeeds:

Attempt 1 → Failed

Attempt 2 → Failed

Attempt 3 → Success

Then the operation completes successfully.

Why limit retries:

Imagine:

Order Service



User Service is down



Retry



Retry



Retry

↓

Retry

↓

Retry

↓

...

This can create unnecessary traffic and put even more load on an already unhealthy service.

Therefore:

Always configure a maximum number of retries.

Retry Delay

You generally shouldn't immediately send another request.

Instead:

Attempt 1

↓

Failed

↓

Wait 500 ms

↓

Attempt 2

For example:

Retry delay = 1 second

Flow:

Attempt 1 → Failed

↓

Wait 1 sec

↓

Attempt 2 → Failed

↓

Wait 1 sec



Attempt 3 → Success

The delay gives the failed service some time to recover.

Exponential Backoff

This is one of the most important concepts.

Instead of using the same delay every time, the delay increases after each failure.

For example:

Attempt 1 → Failed



Wait 1 second

Attempt 2 → Failed



Wait 2 seconds

Attempt 3 → Failed



Wait 4 seconds

Attempt 4 → Failed



Wait 8 seconds

This is called **exponential backoff**.

A simple formula is:

Delay = Initial Delay $\times 2^{(\text{retry number})}$

For example:

Initial delay = 1 second

You could have:

Retry 1 → 1 second

Retry 2 → 2 seconds

Retry 3 → 4 seconds

Retry 4 → 8 seconds

In real systems, **jitter** is often added so that many clients don't retry at exactly the same time.

Step 5 — Implement Retry

Let's understand it.

First request

Order Service

| Attempt 1

User Service

|

X Failed

Because the failure may be temporary, we retry.

Second request

Order Service

| Attempt 2

User Service

|

X Failed

Still failed.

We retry again.

Third request

Order Service

| Attempt 3

User Service

|

✓ Success

Now the request succeeds.

So the final result is:

Order Service → SUCCESS

Step 6 — Configure Retry Limits

We should document three things.

Maximum Attempts

Example:

Maximum attempts = 3

1st attempt

↓

2nd attempt

↓

3rd attempt

↓

STOP

Retryable Errors

These are errors where trying again **may succeed**.

For example:

503 Service Unavailable

may be retryable when the failure is transient.

Another example can be certain temporary network failures.

Conceptually:

503



Wait



Retry

Non-Retryable Errors

Some errors should **not** be retried.

For example:

400 Bad Request

If your request itself is invalid:

Order Service

| Invalid request

User Service



400 Bad Request

Retrying the exact same invalid request will normally produce the same error.

Similarly, an authentication/authorization error such as:

401 Unauthorized

usually needs the underlying authentication problem fixed rather than blindly retrying.

So:

Retryable error



Non-retryable error



Don't retry



Return error

Step 7 — Test Retry

This means we intentionally make the **User Service fail temporarily** so that we can verify whether the retry configuration in the **Order Service** is working correctly.

Architecture

Client



Order Service

| HTTP Request



User Service

Suppose we configure:

Maximum retry attempts = 3

Retry delay = 1 second

Now temporarily make User Service return an error.

For example:

User Service → HTTP 503

The Order Service should behave like this:

Attempt 1



503 Failure



Wait



Attempt 2



503 Failure



Wait



Attempt 3



503 Failure



Stop

We testing:

We want to verify:

1. Does the first request fail?
2. Does Order Service retry?
3. Does it wait according to the configured delay?
4. Does it stop after the maximum attempts?
5. Does it return a proper final error?

Example: Temporary Failure

Suppose User Service is unavailable for a short period.

Attempt 1 → Failed

Attempt 2 → Failed

Attempt 3 → Success

Then:

Order Service



Retry 1



Retry 2



Success

The client receives a successful response.

This demonstrates that retry can recover from a **temporary failure**.

Step 8 — Handle Final Failure

If all retries fail

Suppose we have:

Maximum attempts = 3

and User Service remains unavailable:

Order Service



Retry 1



Retry 2



Retry 3



Final Failure

At this point, Order Service **must stop retrying**.

It should not do this:

Retry



Retry



Retry



Retry



Retry



...

That would be an **infinite retry loop**.

Instead:

Maximum retries reached



Final Failure



Controlled Error Response

Final Failure:

A **final failure** means:

The request failed even after all allowed retry attempts have been used.

For example:

Attempt 1 → 503

Attempt 2 → 503

Attempt 3 → 503

Now:

Final Failure

The application should handle this properly.

Controlled Error Response

Instead of exposing internal technical details to the user, Order Service should return a controlled response.

For example:

HTTP 503 Service Unavailable

Response:

```
{  
  "status": 503,  
  "message": "User Service is temporarily unavailable",  
  "timestamp": "2026-09-22T14:30:00"  
}
```

The exact status and message should match your application's API design.

Controlled Error Response:

Imagine the client receives something like:

```
java.net.SocketTimeoutException
```

Connection refused

Connection pool exhausted

Retry exhausted

That's not a good API response.

Instead:

```
{  
  "status": 503,  
  "message": "Unable to retrieve user information. Please try again later."  
}
```

This is much cleaner.

Step 9 — Add Logging

Request Started

When Order Service starts processing a request:

INFO Request started

Example:

INFO Request started: Create Order

This tells us that the operation has begun.

Retry Attempt

When the first attempt fails and the application tries again:

WARN Retry attempt

Better logging can include the attempt number:

WARN Retry attempt=1

Then:

WARN Retry attempt=2

This helps developers understand how many times the request was retried.

Retry Failure

If a retry fails:

WARN Retry failure

For example:

WARN Retry failure: attempt=1

You might also record useful technical information such as:

service=User-Service

attempt=1

error=ServiceUnavailable

Final Failure

If all retry attempts fail:

ERROR Final failure

For example:

ERROR Final failure: User Service unavailable

This tells us:

The application used all allowed retry attempts and still could not complete the operation.

Complete Example of Logs

Suppose we configure:

Maximum attempts = 3

User Service is unavailable.

The logs could look like:

INFO Request started

WARN Retry attempt=1

WARN Retry failure=1

WARN Retry attempt=2

WARN Retry failure=2

WARN Retry attempt=3

ERROR Final failure

This is much easier to troubleshoot than simply seeing:

ERROR Request failed

Do NOT Log Passwords or Secrets

Do not log passwords, tokens or other secrets.

This is extremely important.

 **Never do this**

INFO username=madhav

INFO password=MyPassword123

INFO token=eyJhbGciOi...

Passwords, access tokens, API keys, session IDs, and similar credentials should not appear in normal application logs.

 **Instead**

Log safe information:

INFO Request started

INFO userId=101

INFO service=User-Service

INFO retryAttempt=2

ERROR User-Service unavailable

You can log identifiers useful for troubleshooting, but avoid sensitive data.

Step 10 — Document

Timeout Configuration

Example:

Connection timeout: 2 seconds

Read timeout: 3 seconds

Retry Configuration

Example:

Maximum attempts: 3

Initial retry delay: 1 second

Backoff: Exponential

Retryable Errors

Document errors that can be retried.

For example:

503 Service Unavailable

Temporary network failure

Connection failure

The exact retryable conditions should be based on your application's client and failure behavior.

Non-Retryable Errors

Document errors that should normally not be retried.

For example:

400 Bad Request

401 Unauthorized

403 Forbidden

A retry won't normally fix an invalid request or missing authorization.

Final Failure Handling

Explain what happens when all retries fail.

For example:

All retry attempts failed

↓

Stop retrying

↓

Log final failure

↓

Return controlled error

↓

HTTP 503